

The Living Map: Spatial Memory for Emergency Robots

IEEE TSYP14 Technical Challenge (RAS × AESS Tunisia) – Phase 1 Short Technical Report

Team: Winek?

Code, simulation and documentation: <https://github.com/BechirSinko/living-map>

1. Problem and approach

When a robot explores a mine, its knowledge disappears if it fails or loses power, and no GPS or network reaches inside. We target the **Mines / Tunnels** environment (mostly 1-D corridors with branches and a single entrance) and detect two event types: **hazardous gas** and **victim detected** (a third, *blocked path*, is reserved in the packet format).

Key idea: the memory lives in the beacons, not in the robot. A *Writer* robot explores autonomously and drops small RF beacons at every event, every junction and every 10 m, forming a chain linked by *previous-beacon* pointers. When it finishes or runs low on battery it returns to the entrance and uploads its log to the Outside Network Area (ONA). If the Writer dies, the beacons still hold the map. A later *Executor* robot is briefed at the entrance and follows the beacons to continue the mission instead of starting from zero.

2. System architecture

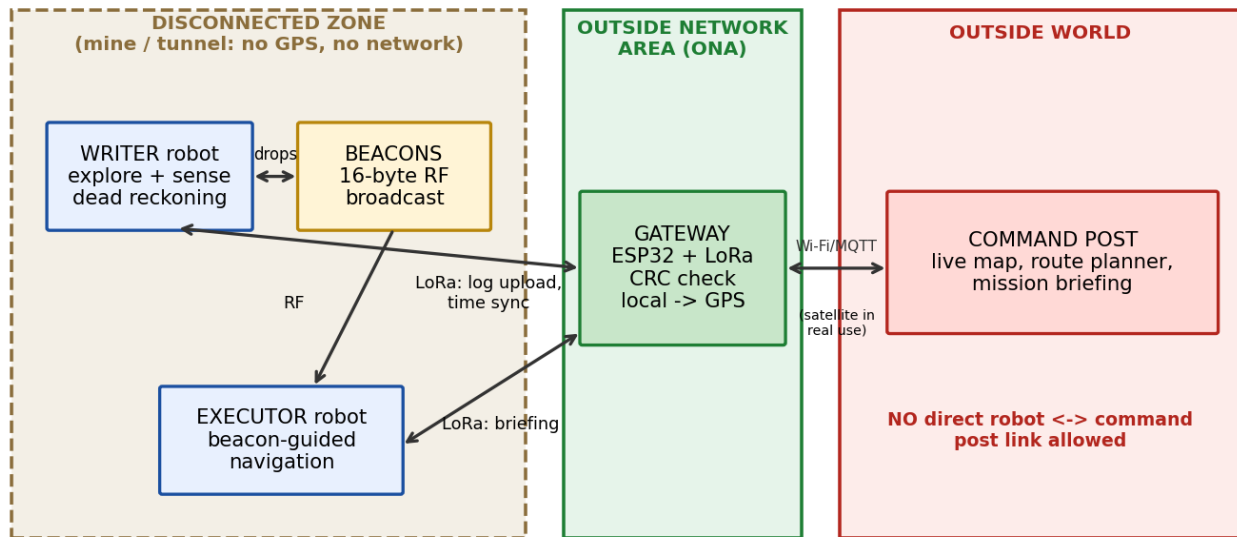


Figure 1: Architecture. Everything between the mine and the outside passes through the ONA gateway.

Hard rule respected: the robots never talk to the command post. Writer and Executor use LoRa (SX1276 on ESP32, 868 MHz band with 433 MHz as fallback; the legal band in Tunisia is being confirmed with the regulator) to the ONA gateway only; the gateway (ESP32 + LoRa) forwards to the command post over Wi-Fi/MQTT, with satellite as the real-deployment option. Beacons are dumb one-way broadcasters: they do not relay, which keeps them cheap and long-lived.

3. Data flow

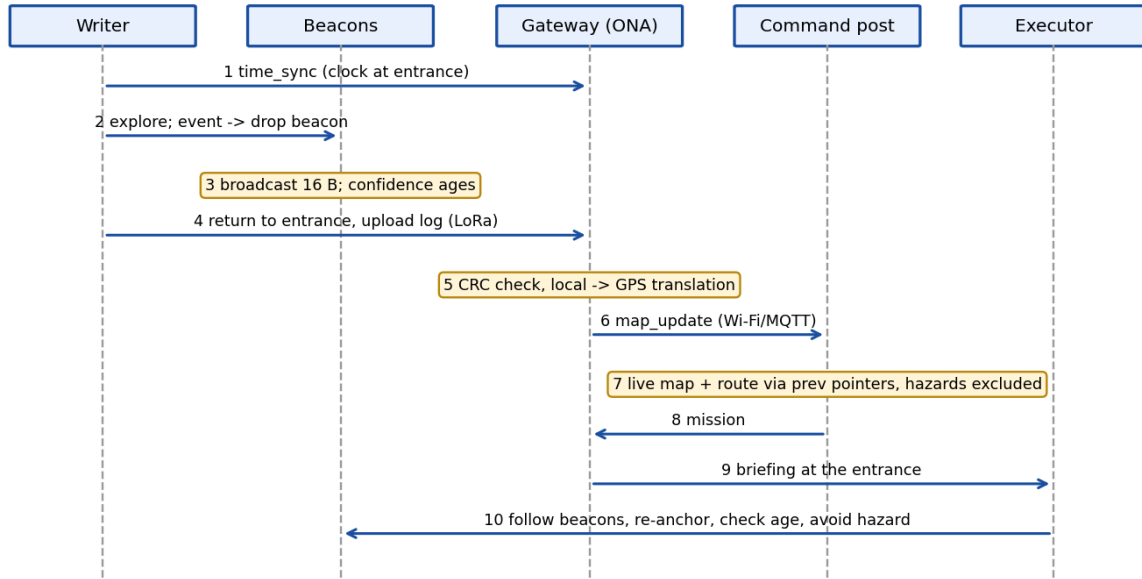


Figure 2: End-to-end data flow, implemented in the simulation message bus.

4. Writer autonomy, event detection and beacon deposition

The Writer explores by **frontier-based depth-first search** over the corridor graph: at each junction it takes an unvisited branch, backtracks at dead ends, and keeps a private frame (origin at the entrance, $+x$ in its starting direction). It has **no GPS**: position comes from dead reckoning (wheel odometry plus IMU), so it drifts; the simulation models a heading bias with random walk and a 1 % scale error.

What to record, and when. A beacon is dropped (a) at the entrance, (b) at every junction or dead end, (c) at every *victim* or *gas* detection, and (d) every 10 m of corridor so gaps stay short. On gas detection (sensor range 5 m) the Writer drops a hazard beacon *before* the plume and turns back; it never enters the hazard zone. The drop mechanism on the physical robot is a servo-based dropper (Phase 2).

5. Beacon message and signal design

A 16-byte big-endian packet carries what was found, where, and when it was written:

Byte	Field	Meaning
0	beacon_id	1..254, assigned sequentially
1	type_flags	high nibble: event (0 waypoint, 1 gas, 2 victim, 3 blocked); low nibble flags: 1 junction, 2 dead end, 4 hazard ahead, 8 entrance
2 / 3	prev_id / next_id	chain pointers (0xFF = none). The simulation uses prev_id; next_id stays 0xFF because a beacon cannot know its successor when dropped
4–7	x, y	int16 centimetres in the Writer frame
8–11	timestamp	uint32 Unix seconds (Writer clock synced from the gateway at the entrance)
12 / 13	confidence / value	uint8 initial confidence; event value (e.g. gas level)
14–15	CRC-16	CCITT-FALSE over bytes 0..13

Prototype variant. The firmware currently uses a compact 14-byte layout (beacon ID, event type, next-hop ID, x , y , bearing, age). It replaces the absolute timestamp by an age counter kept inside the beacon, which needs no clock synchronisation; the decay law is the same. The two layouts will be unified in Phase 2.

Radio link parameters (identical on every node: Writer, Executor, beacons, gateway):

Setting	Value	Rationale
Frequency	868 MHz band (433 MHz fallback)	final choice depends on the Tunisian regulation, being confirmed
Spreading factor	7	shortest airtime; the corridor distances are short
Bandwidth	125 kHz	standard LoRa setting
Coding rate	4/5	least overhead; corruption is caught by our own CRC-16
Sync word	0x12 (private)	isolates our network from LoRaWAN traffic
Preamble	8 symbols	standard
TX power	14 dBm	about 25 mW; the Writer programs beacons at 0 dBm so that only the beacons in its tube hear it

With these settings the symbol time is $T_s = 2^7/125 \text{ kHz} = 1.024 \text{ ms}$. The 16-byte payload (explicit header, radio CRC on) takes 12.5 ms of preamble plus 38 payload symbols (38.9 ms), so the **airtime is about 51.5 ms per packet**. A 1 % duty-cycle limit therefore allows one transmission every 5.2 s per node at most; beacons rebroadcast every 5.5 s plus a random jitter of 0–2 s ($\approx 0.8 \%$ duty cycle). With three beacons in range of one another, the chance that a packet overlaps another is about 3 % ($1 - e^{-2(n-1)T/I}$ with $T = 51.5 \text{ ms}$ and $I = 6.5 \text{ s}$), and a missed beacon is heard on its next cycle. The receiver sensitivity at SF7/125 kHz is about -124.5 dBm (thermal noise, 6 dB noise figure assumed, -7.5 dB required SNR).

Message aging. A beacon cannot be updated, so trust decays with age:

$$\text{confidence} = c_0 \cdot 0.5^{\text{age}/T_{1/2}},$$

with half-lives $T_{1/2}$: gas 10 min, blocked path 30 min, victim 60 min, waypoint 24 h. Below 0.25 the data is flagged *stale: verify before trusting*. In the simulation the gas beacon is already at 0.24 after a 15-minute delay, so the Executor treats it as a hazard to verify, while the victim beacon is still at 0.79.

6. Frame translation

Robots only use their private local frame (metres, origin at the entrance). The **gateway** translates to GPS using an anchor $(\varphi_0, \lambda_0, \psi)$, where ψ is the compass bearing of $+x$ clockwise from north:

$$E = x \sin \psi - y \cos \psi, \quad N = x \cos \psi + y \sin \psi,$$

then E and N are divided by the WGS-84 metres-per-degree at φ_0 . Error sources are dead-reckoning drift, heading error (1° is about 1.7 m per 100 m) and entrance GPS error. GPS is used only for the displayed map, so navigation is unaffected by an anchor error.

7. Outside Network Area and Executor robot

ONA. The gateway (ESP32 + SX1276) sits at the entrance, within GPS and network coverage. It receives LoRa packets, checks the CRC, translates coordinates, forwards a map update to the command post (Wi-Fi/MQTT; satellite for real deployments), and relays the mission back. The command post shows the live map and plans the route; in the simulation a direct robot-to-command-post message raises an error.

Executor. Briefed at the entrance (mission: target beacon, route of beacon IDs and coordinates, hazards to avoid), it navigates beacon to beacon: at each junction it picks the branch closest to the bearing of the next beacon, **re-anchors** its position to the beacon coordinates on every pass (stopping drift accumulation), checks each beacon’s age and CRC, and never enters a hazard-flagged branch. The route is computed by following *prev* pointers from the victim beacon back to the entrance, so a gas beacon (a side leaf) is excluded automatically.

8. Hardware prototype

Every node uses the same ESP32 + SX1276 pair driven by RadioLib, so one library and one wiring serve all nodes.

Node	Qty	Role and main parts
Beacon	3 (+1 spare)	stores one message, repeats it, deep-sleeps in between; ESP32, SX1276, 1000 mAh LiPo
Gateway (ONA)	1	LoRa receive, $(x, y) \rightarrow (\text{lat}, \text{lon})$, Wi-Fi/MQTT forward with a 32-message store-and-forward queue, briefs the Executor; power-bank backup
Writer	1	explores and senses: MQ gas sensor, AMG8833 thermal array (victims), VL53L0X distance sensors, wheel encoders + MPU6050 for dead reckoning; programs and drops beacons
Executor	1	receives the mission before entry, follows the beacon chain; same radio, motors and encoders

Drop mechanism. A vertical tube holds the beacons in ID order and a servo-driven trapdoor releases one at a time. The Writer first sends a WRITE packet addressed to the next beacon ID (three times, at 0 dBm), then opens the trapdoor for 400 ms. It then expects to hear the beacon’s first broadcast within about 10 s; otherwise it retries once and logs a failure. Beacons are pre-numbered in drop order, so each one can point to the next, and the Executor can follow the chain inward from the first beacon with no log at all. The prototype briefing is a 4-byte message (first beacon ID, target event).

Power. A programmed beacon is awake 0.15 s per cycle at about 125 mA and asleep at about 0.05 mA, giving an average of 3.8 mA, i.e. about 263 h (11 days) on 1000 mAh. These currents are datasheet-based assumptions to be measured on the bench. The prototype uses 3 beacons, while the simulation drops 13 to exercise a longer mine.

9. Simulation results

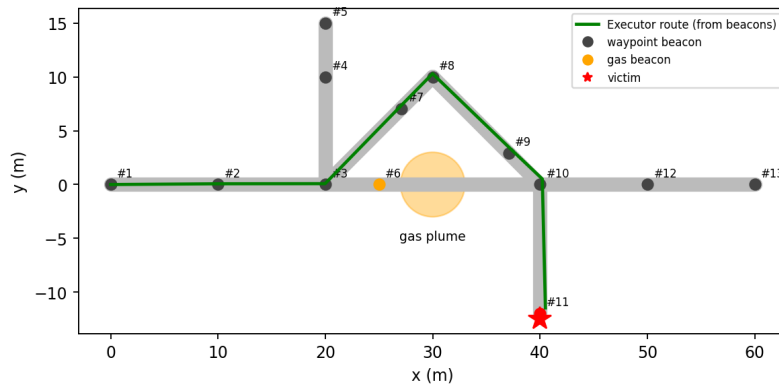


Figure 3: Simulated mine: loop with a gas plume on the direct corridor, a dead end, and a victim branch.

Metric (seed 7)	Result
Beacons dropped	13 (waypoints, 1 gas, 1 victim)
Writer drift on return to entrance	1.38 m after about 250 m of travel
Route chosen by command post	beacons 1-2-3-7-8-9-10-11; gas beacon 6 excluded, bypass used instead of the direct corridor
Executor delay / result	deployed 15 min later, reached the victim beacon
Executor gas exposure	0 steps; CRC errors injected at 10 % were rejected and retried

Run it with `py run_demo.py`; the automated tests (`py -m pytest`) check mission success, zero gas exposure and that no direct link to the command post exists. The simulation uses straight corridors on a graph and a simplified sensor model; obstacle avoidance and RF propagation are left for the physical prototype.

10. Innovation

1. **Memory in the environment.** Knowledge survives the robot: beacons are pre-numbered in drop order and each points to the next, so the Executor can follow the chain inward from the first beacon even if the Writer never returns. (The simulation routes through the command post with *prev* pointers; the prototype firmware uses next-hop pointers.)
2. **Self-describing, aging beacons.** Each 16-byte packet carries type, position, time and a decaying confidence, so stale hazard data is flagged instead of blindly trusted.
3. **Drift-free relay of position.** The Executor re-anchors on every beacon, so the Writer’s dead-reckoning errors do not accumulate over a mission.
4. **Frame separation.** Robots never use GPS; only the gateway translates, so a wrong GPS anchor degrades only the displayed map.
5. **Low-cost, dumb beacons** that only broadcast, keeping the hardware cheap and the system robust.

11. Failure cases and implementation plan (summary)

Failure cases (full table in `docs/failure_cases.md`): Writer dies or is stuck (beacons still hold the map); beacon lost or dead (bounded search, then dead reckoning to the next known beacon); RF collisions and CRC errors (jitter, retry, CRC drop); stale data (aging and re-sensing); dead-reckoning drift (re-anchoring); bad GPS anchor or heading (only the map is affected); unsynchronised clock; gateway or uplink down (buffering, backup uplink); hazard blocking the only route (command post reports no safe route).

Implementation plan (full plan in `docs/implementation_plan.md`): two roles, a software lead (packet, frames, command post, gateway software, documentation) and an embedded lead (firmware, LoRa, sensors, chassis, beacon dropper). Eight weeks to 01/12/2026: parts and LoRa link (weeks 1–2), Writer base, command post and event sensing (3–4), Executor navigation (5), full integration (6), failure tests (7), documentation and pitch (8), with the riskiest items scheduled first.